Reviewer Pack — Sandpile p-Rank Lower Bound for ACC^0 Circuits Approximating…

Entry 496534d5bd8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 18:01:04 UTC

Sandpile p-Rank Lower Bound for ACC^0 Circuits Approximating Parity

Entry ID: `496534d5bd8f` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-19 18:01:04 UTC

1. Conjecture

Field A (mathematical branch): Sandpile groups / chip-firing on multigraphs (critical group theory via Smith normal form of the discrete Laplacian) **Field B** (complexity object): AC^0[MOD_p] circuit structure approximating PARITY (Williams-style ACC^0 lower bounds)

Statement:

> Let C be any AC^0[MOD_p] circuit (p an odd prime ≥ 3) of size s and depth d on n inputs that ε-approximates PARITY_n, i.e. Pr_{x∈{0,1}^n}[C(x)=PARITY(x)] ≥ 1/2 + ε. Construct the underlying multigraph G_C by giving each MOD_p gate p parallel edges to each of its inputs, each AND/OR/NOT gate one edge per input, and each circuit input one edge to a common 'sink' node. Then the 2-rank of the sandpile (critical) group K(G_C) = Z^{|V|-1}/imL̃ satisfies rk_2(K(G_C)) ≥ c · ε · n^{1/d} / log(s+1) for an absolute constant c > 0; equivalently, any circuit with sandpile 2-rank below this threshold cannot ε-approximate PARITY.

Rationale (proposer's reasoning):

> Sandpile p-rank is a *syntactic, relational* invariant of the circuit DAG (computed from the Laplacian's Smith normal form, not from the truth table), so it sidesteps Razborov–Rudich naturalness. Smith normal form over Z is fundamentally a torsion/integer-lattice operation, with no clean lift to a polynomial extension of the Boolean ring, so it dodges Aaronson–Wigderson algebrization. The conjecture leverages the known fact that AC^0[MOD_p] is Smolensky-obstructed against PARITY by *modular-algebraic* reasons, and proposes that the obstruction reveals itself as torsion in the spanning-tree module of the circuit's multigraph.

Taxonomy category: `ACC_LB_via_WILLIAMS` (status at proposal time: partially_alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `b3277e181cce8441`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 30 random AC^0[MOD_3] circuits per (n∈{8,10,12,14}, d∈{2,3}, s∈{2n,4n,8n}) with ε≥0.02, compute R=rk_2(K(G_C))·log(s+1)/(ε·n^{1/d}). SUPPORTED iff min R≥0.01 over all such instances AND median R≥0.1. FALSIFIED iff any instance with ε≥0.1 yields R<0.01.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.95	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `sandpile group critical group circuit complexity Laplacian rank` - `ACC0 MOD_p PARITY approximation lower bound algebraic` - `Smith normal form Laplacian Boolean circuit AC0 lower bound`

Top relevant hits considered: - [http://arxiv.org/abs/hep-ph/0610012v1] Tevatron-for-LHC Report of the QCD Working Group - [http://arxiv.org/abs/1908.06409v1] Schur multipliers of special p-groups of rank 2 - [http://arxiv.org/abs/math/0608534v2] On Automorphisms of Finite p-groups - [http://arxiv.org/abs/1412.3178v3] Some approximation problems in semi-algebraic geometry - [http://arxiv.org/abs/0906.0693v3] An improved lower bound on the counterfeit coins problem - [http://arxiv.org/abs/1007.1875v2] Lower Bounds for Quantum Oblivious Transfer - [http://arxiv.org/abs/2304.02770v2] Tight Correlation Bounds for Circuits Between AC0 and TC0 - [http://arxiv.org/abs/2407.04826v1] Multi-strategy Based Quantum Cost Reduction of Quantum Boolean Circuits

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
import math
from fractions import Fraction

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m = len(A)
    n = len(B[0])
```

```
    p = len(B)
    C = [[0 for _ in range(n)] for _ in range(m)]
    for i in range(m):
        for j in range(n):
            for k in range(p):
                C[i][j] += A[i][k] * B[k][j]
    return C

def gaussian_elimination(A, b):
    m = len(A)
    n = len(A[0])
    augmented = [A[i] + [b[i]] for i in range(m)]
    for i in range(n):
        max_row = i
        for j in range(i+1, m):
            if abs(augmented[j][i]) > abs(augmented[max_row][i]):
                max_row = j
        augmented[i], augmented[max_row] = augmented[max_row], augmented[i]
        pivot = augmented[i][i]
        for j in range(n + 1):
            augmented[i][j] /= pivot
        for j in range(m):
            if j != i:
                factor = augmented[j][i]
                for k in range(n + 1):
                    augmented[j][k] -= factor * augmented[i][k]
    return [row[-1] for row in augmented]

def rank(A):
    m = len(A)
    n = len(A[0])
    reduced = gaussian_elimination(A, [0] * m)
    return sum(1 for x in reduced if any(x[j] != 0 for j in range(n)))

def generate_ac0_mod_p_circuit(n, d, s):
    gates = ['MOD_3', 'AND', 'OR', 'NOT']
    inputs = list(range(n))
    outputs = [n + i for i in range(s)]

    circuit = []
    for _ in range(s):
        gate_type = random.choice(gates)
        if gate_type == 'MOD_3':
            inputs_used = random.sample(inputs, 2)
            circuit.append((gate_type, inputs_used))
        elif gate_type == 'AND' or gate_type == 'OR':
            inputs_used = random.sample(inputs, 1)
            circuit.append((gate_type, inputs_used))
        elif gate_type == 'NOT':
            input_used = random.choice(inputs)
            circuit.append((gate_type, [input_used]))

    return circuit, outputs

def build_multigraph(circuit, outputs):
    V = set()
```

```
    E = []

    for gate, inputs in circuit:
        if gate == 'MOD_3':
            for _ in range(3):
                for input_node in inputs:
                    V.add(input_node)
                    E.append((input_node, gate))
        elif gate == 'AND' or gate == 'OR':
            for input_node in inputs:
                V.add(input_node)
                E.append((input_node, gate))
        elif gate == 'NOT':
            V.add(inputs[0])
            E.append((inputs[0], gate))

    for output in outputs:
        V.add(output)
        E.append((output, 'sink'))

    return V, E

def compute_epsilon(circuit, outputs):
    n = len(outputs)
    count = 0
    for x in range(2**n):
        input_bits = [int(b) for b in format(x, f'0{n}b')]
        output_bits = [circuit[i][1] if circuit[i][0] == 'NOT' else input_bits[circuit[i][1]] for
        predicted_output = sum(output_bits) % 2
        actual_output = x % 2
        count += int(predicted_output == actual_output)
    return abs(count / (2**n - 0.5))

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n_values = [8, 10, 12, 14]
    d_values = [2, 3]
    s_values = [2 * n, 4 * n, 8 * n]

    results = []
    for n in n_values:
        for d in d_values:
            for s in s_values:
                circuit, outputs = generate_ac0_mod_p_circuit(n, d, s)
                epsilon = compute_epsilon(circuit, outputs)
                if epsilon < 0.02:
                    continue

                V, E = build_multigraph(circuit, outputs)
                L_tilde = []
                for i in range(len(V) - 1):
                    row = [0] * (len(V) - 1)
                    for j in range(len(E)):
                        if E[j][0] == V[i]:
                            row[V.index(E[j][1])] += 1
```

```
                    elif E[j][1] == V[i]:
                        row[V.index(E[j][0])] -= 1
                L_tilde.append(row)

            rank_L_tilde_mod_2 = rank(L_tilde)
            R = (len(V) - 1 - rank_L_tilde_mod_2) * math.log(s + 1) / (epsilon * n**(1/d))
            results.append(R)

    if not results:
        return {
            "metric_name": "R",
            "metric_value": None,
            "instances_tested": 0,
            "conjecture_holds": False,
            "counterexample": "no_valid_instances"
        }

    min_R = min(results)
    median_R = sorted(results)[len(results) // 2]

    return {
        "metric_name": "R",
        "metric_value": R,
        "instances_tested": len(results),
        "conjecture_holds": min_R >= 0.01 and median_R >= 0.1,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]]
    if not seeds:
        seeds = [3, 5, 7, 11, 13, 17, 19, 23, 29, 31] * 3

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        if "metric_value" in result and result["metric_value"] is not None:
            results.append(result["metric_value"])

    if all(r is not None for r in results):
        mean_R = sum(results) / len(results)
        std_R = math.sqrt(sum((r - mean_R)**2 for r in results) / len(results))
        support_fraction = sum(1 for r in results if r >= 0.01 and r >= 0.1) / len(results)

        if support_fraction >= 0.8:
            print(f"RESULT: SUPPORTED mean={mean_R} std={std_R} support_fraction={support_fraction
        elif any(r < 0.01 for r in results):
            first_failing_seed = seeds[results.index(min([r for r in results if r < 0.01]))]
            print(f"RESULT: FALSIFIED counterexample=\"R<{0.01}\" first_failing_seed={first_failin
        else:
            print("RESULT: INCONCLUSIVE not_enough_valid_instances")
    else:
        print("RESULT: INCONCLUSIVE missing_data")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44e7f4da.py", line 176, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_44e7f4da.py", line 123, in run_trial
    s_values = [2 * n, 4 * n, 8 * n]
                    ^
UnboundLocalError: cannot access local variable 'n' where it is not associated with a value
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed with an UnboundLocalError before any trials ran, so no data on R was collected and neither the support nor falsification condition can be evaluated. | next: Fix the scoping bug so ‘n’ is defined before constructing s_values (e.g., move s_values inside the per-(n,d) loop), then rerun the pre-registered sweep.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	225465
2	preregistration	claude_max	opus	0	0	5679
3	novelty	claude_max	opus	0	0	4143
4	novelty	claude_max	opus	0	0	7309
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19232
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16417
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19401
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	19507
9	judge	claude_max	opus	0	0	5028

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 322181 ms total latency. Provider mix: {‘claude_max’: 5, ‘ollama_remote’: 4}

(full prompt+response transcripts available in `research/audit/496534d5bd8f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/496534d5bd8f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/496534d5bd8f.tar.gz` (if generated)